UNIVERSIDAD NACIONAL DE LOJA
Facultad de la Energía, las Industrias y los Recursos Naturales no Renovables

Especificación de requisitos de software

Proyecto: Prototipo SmartPark

Desarrollo de un prototipo web para la clasificación automática del estado de ocupación de estacionamientos integrando el modelo YOLOv11

Proyecto de Integración Curricular

Elaborado por:	Saúl Alejandro Ludeña Ocampo
Director:	Ing. Edwin René Guamán Quinche
Fecha de revisión	20 de abril del 2026
Aprobado por:	Ing. Edwin René Guamán Quinche
Fecha de aprobación:	07 de mayo del 2026

Edwin René Guamán Quinche

ASESOR

1. Introducción

1.1 Propósito

El presente documento tiene como propósito definir, de manera formal y detallada, los requisitos funcionales y no funcionales del prototipo web SmartPark para la clasificación automática del estado de ocupación de estacionamientos. Los requisitos funcionales describen las capacidades observables del sistema desde la perspectiva del usuario, mientras que los requisitos no funcionales establecen los criterios mínimos de rendimiento esperados del modelo en términos de exactitud, precisión y sensibilidad. Esta especificación constituye la guía fundamental para las fases de diseño y codificación, asegurando la trazabilidad de los requisitos durante todo el ciclo de vida del desarrollo.

1.2 Alcance

1.2.1 Identificación del producto.

El producto de software objeto de esta especificación se denomina "Prototipo SmartPark". Se define como un prototipo web basado en visión por computador diseñado para automatizar el monitoreo de plazas de estacionamiento.

El sistema está compuesto por dos módulos principales:

- Módulo de Inferencia (Back-end): Desarrollado en Python con el framework Flask, encargado de procesar las imágenes mediante el modelo YOLOv11.
- Interfaz de Gestión Visual (Front-end): Una aplicación web desarrollada en Next.js que permite la interacción del usuario con los resultados del análisis.

1.2.2 Funcionalidades.

El sistema permitirá:

- La carga manual de imágenes en formatos estándar (.webp, .jpg, .jpeg y .png).
- Proporcionar herramientas para que el usuario delimite manualmente las coordenadas de las plazas de estacionamiento.
- Proporcionar herramientas para la delimitación automática de las plazas de estacionamientos mediante el servicio de inferencia en la nube RoboFlow.
- Ejecutar algoritmos de procesamiento para optimizar la calidad de la imagen antes de la detección.
- Identificar y clasificar el estado de ocupación (Libre/Ocupado) de cada espacio delimitado.
- Presentar un reporte visual y estadístico del estado del estacionamiento.

El software no permitirá:

- Procesamiento de video en tiempo real.
- Gestionar el cobro o facturación de los estacionamientos.

- Reconocimiento de placas vehiculares (LPR).
- Controlar dispositivos físicos como barreras o sensores IoT.

1.2.3 Beneficios y metas.

Beneficios:

- Optimización del flujo de trabajo: Reduce el tiempo empleado en la configuración y delimitación del límite de las plazas de estacionamiento.
- Interpretabilidad de datos: Facilita la comprensión de los resultados de la inferencia.

La meta principal es desarrollar una herramienta que integre la potencia de detección de YOLOv11 con una arquitectura web. Se busca demostrar que es posible la construcción de un sistema de clasificación de alta precisión a bajo costo que sirva como base para futuras implementaciones de movilidad urbana en el marco de las Smart Cities.

1.3 Acrónimos y abreviaturas

La siguiente tabla recoge, en orden alfabético, todos los términos, acrónimos y abreviaturas utilizados en el presente documento.

Tabla 1. Acrónimos y abreviaturas

Término	Definición en el contexto del documento de especificación de requisitos
API	Interfaz de comunicación entre el front-end (Next.js) y el back-end (Flask) mediante solicitudes HTTP.
Back-end	Módulo de inferencia, desarrollado en Python con el framework Flask.
Bounding box	Rectángulo delimitador generado por YOLOv11 que envuelve el área de un vehículo detectado. Representado con coordenadas (x_min, y_min, x_max, y_max).
Plaza de estacionamiento	Espacio físico destinado al estacionamiento de un único vehículo, definida mediante una ROI.
Canvas / Lienzo	Área de dibujo interactiva de la interfaz web sobre la que el usuario renderiza la imagen cargada y traza los polígonos que representan cada plaza de estacionamiento.
Centroide	Punto central del bounding box de un vehículo, calculado como ((x_min + x_max) / 2, (y_min + y_max) / 2).
CLAHE	Algoritmo de mejora de contraste adaptativo aplicado durante el procesamiento para mitigar efectos de sombras y variaciones lumínicas.
COCO	Dataset de referencia de visión computacional.
Confianza (score)	Valor entre 0 y 1 asignado por YOLOv11 a cada detección, indicando la probabilidad de pertenencia a la clase predicha. Se aplica un umbral configurable con un valor por defecto de 0.45.

Drag & Drop	Mecanismo de carga de archivos mediante el cual el usuario arrastra una imagen desde su sistema de archivos hasta la zona designada en la interfaz.
Front-end	Módulo de interfaz de gestión visual implementado con Next.js.
Letterboxing	Técnica de reescalado que preserva la relación de aspecto original al redimensionar una imagen a 640×640 px, añadiendo relleno gris en los bordes sobrantes.
Next.js	Framework de React, provee la interfaz reactiva para carga de imágenes, delimitación de ROIs y visualización de resultados.
Overlay	Capa semitransparente renderizada sobre la imagen original para indicar el estado de cada cajón.
Procesamiento de imagen	Conjunto de transformaciones aplicadas a la imagen antes de la inferencia: reescalado a 640×640 px mediante Letterboxing, normalización de píxeles y aplicación del algoritmo CLAHE y filtro Gaussiano.
RF	Descripción de una capacidad observable del sistema desde la perspectiva del usuario.
ROI	Polígono trazado manualmente por el usuario sobre el lienzo para delimitar una plaza de estacionamiento individual.
YOLOv11	Motor de inferencia del prototipo Última versión de la serie Ultralytics YOLO, también denominada YOLO11. Detecta objetos en la imagen y genera bounding boxes con score de confianza para cada vehículo identificado.
PKLOT	Conjunto de datos robusto y ampliamente utilizado en investigación de visión por computadora para la clasificación y detección de espacios de estacionamiento.

1.4 Referencias

- P. R. L. de Almeida, L. S. Oliveira, E. Silva Jr., A. Britto Jr., and A. Koerich, "PKLot – A robust dataset for parking lot classification," *Expert Systems with Applications*, vol. 42, no. 11, pp. 4937–4949, 2015. doi: 10.1016/j.eswa.2015.02.009.

2. Descripción General

2.1 Perspectiva del Producto

El prototipo SmartPark se define como un producto de software independiente. Su desarrollo responde a la necesidad de automatizar la clasificación del estado de ocupación de las plazas de estacionamiento mediante visión por computador, operando de manera autocontenida en un entorno de ejecución local.

El prototipo se estructura bajo una arquitectura cliente-servidor compuesta por dos módulos principales: el módulo de interfaz de gestión visual y el módulo de inferencia. La comunicación entre ambos se realiza mediante solicitudes HTTP a través de una API REST. Complementariamente, el sistema integra el servicio externo de RoboFlow mediante su API alojada para la detección automática de los límites de las plazas de estacionamiento, constituyendo la única dependencia externa del prototipo.

2.2 Funciones del Producto

Las funciones principales incluyen:

- Gestión de carga de archivos de imagen.
- Herramienta de delimitación manual de plazas de estacionamiento.
- Herramientas de delimitación automática de plazas de estacionamiento.
- Ejecución de algoritmos de procesamiento de imagen.
- Inferencia mediante el modelo YOLOv11.
- Visualización gráfica del estado de ocupación inmediato sobre la imagen.
- Reporte de métricas del estado del estacionamiento.

2.3 Características de los Usuarios

El software está destinado a:

a. Investigadores en Visión Artificial y Aprendizaje Profundo:
   El software les permite:
   - Realizar evaluaciones comparativas de las métricas de precisión, sensibilidad y exactitud de YOLOv11 frente a otros modelos.
   - Utilizar el sistema como un entorno de experimentación controlado para validar la robustez del modelo ante variables externas, tales como cambios en las condiciones lumínicas, oclusiones parciales o variaciones en la resolución de las imágenes capturadas.

b. Desarrolladores:
   Dirigido a ingenieros que buscan soluciones de bajo costo y alta escalabilidad para la gestión de infraestructuras urbanas. El software puede resultar útil para:

   - Demostrar la viabilidad de la monitorización automatizada de plazas de estacionamiento sin depender de sensores físicos.

- Punto de partida para la integración de nuevas funcionalidades.

2.4 Restricciones

El sistema opera bajo las siguientes restricciones de hardware y software:

Hardware: El equipo de ejecución debe disponer de capacidad suficiente para ejecutar inferencias de visión artificial, con soporte para procesamiento en CPU o GPU

Software:

- Sistema operativo: Ubuntu 20.04 o superior.
- Python 3.10 o superior.
- Node.js 18 o superior.
- Las siguientes bibliotecas de Python deben estar instaladas en las versiones mínimas especificadas: Flask 3.1.1, Ultralytics 8.3.0, OpenCV 4.9.0, Pillow 10.3.0 y NumPy 1.26.4.

Conectividad: El sistema requiere conexión a internet para la funcionalidad de delimitación automática de plazas, dado que esta depende del servicio de inferencia en la nube de Roboflow.

2.5 Suposiciones y dependencias

Suposiciones:

El correcto funcionamiento del prototipo SmartPark está sujeto a las siguientes suposiciones sobre el entorno de uso y las características de los datos de entrada:

- Se asume que el usuario dispone de imágenes estáticas de estacionamientos capturadas previamente.
- Se asume que las imágenes de entrada han sido capturadas desde una perspectiva cenital o semicenital, condición necesaria para que la delimitación de plazas y la detección de vehículos produzcan resultados confiables.
- Se asume que las imágenes de entrada tienen una resolución mínima de 320×240 píxeles.
- Se asume que los vehículos presentes en las imágenes pertenecen a alguna de las cuatro categorías vehiculares del dataset COCO empleadas por el modelo: automóvil, camión, autobús y motocicleta.

Dependencias:

- El módulo de delimitación automática de plazas depende de la disponibilidad del servicio de inferencia en la nube de Roboflow.

- El modelo YOLOv11 depende de los pesos preentrenados proporcionados por Ultralytics. Cualquier cambio en la disponibilidad o versión de dichos pesos puede afectar el comportamiento del sistema.
- El frontend depende de la disponibilidad del servidor backend en ejecución local para procesar las solicitudes de inferencia.

3. Requisitos Específicos

3.1 Interfaces Externas

3.1.1 Interfaces de Usuario

Pantalla correspondiente a la carga de imágen

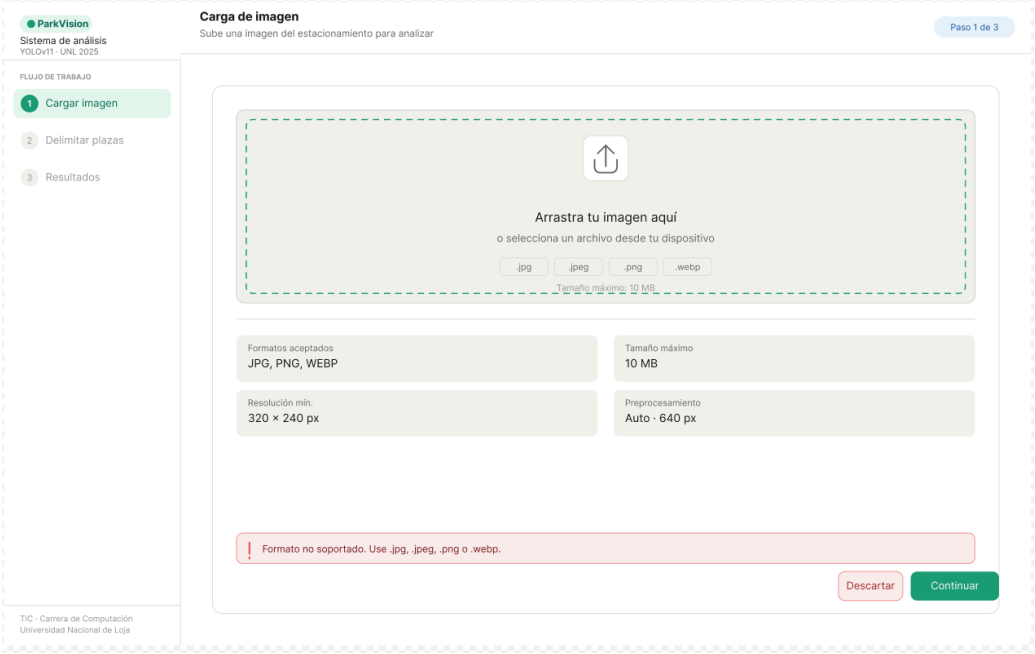

Figura 1: Interfaz carga de imágenes

Pantalla correspondiente al editor de regiones de interés

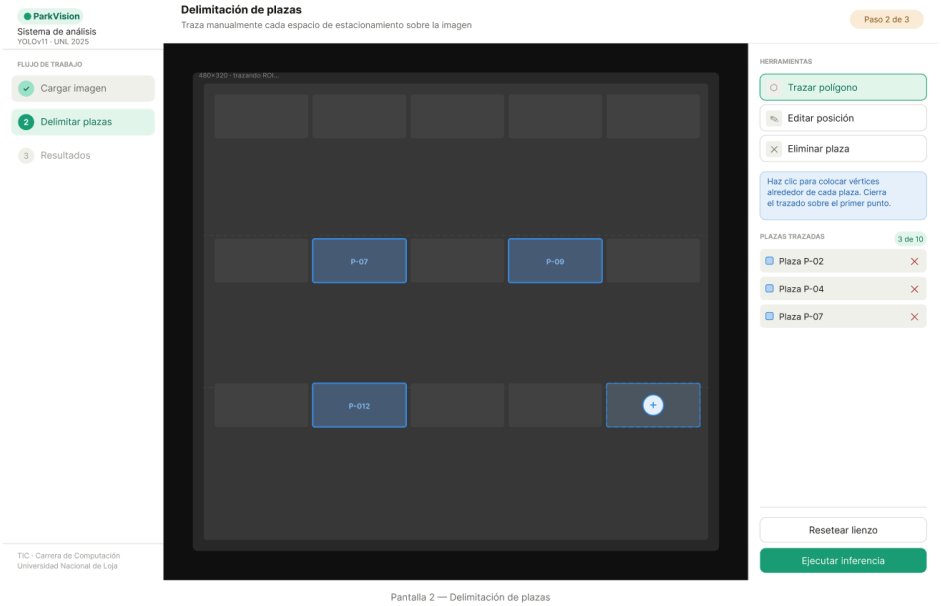

Figura 2: Editor de ROI

Pantalla correspondiente a la visualización de resultados

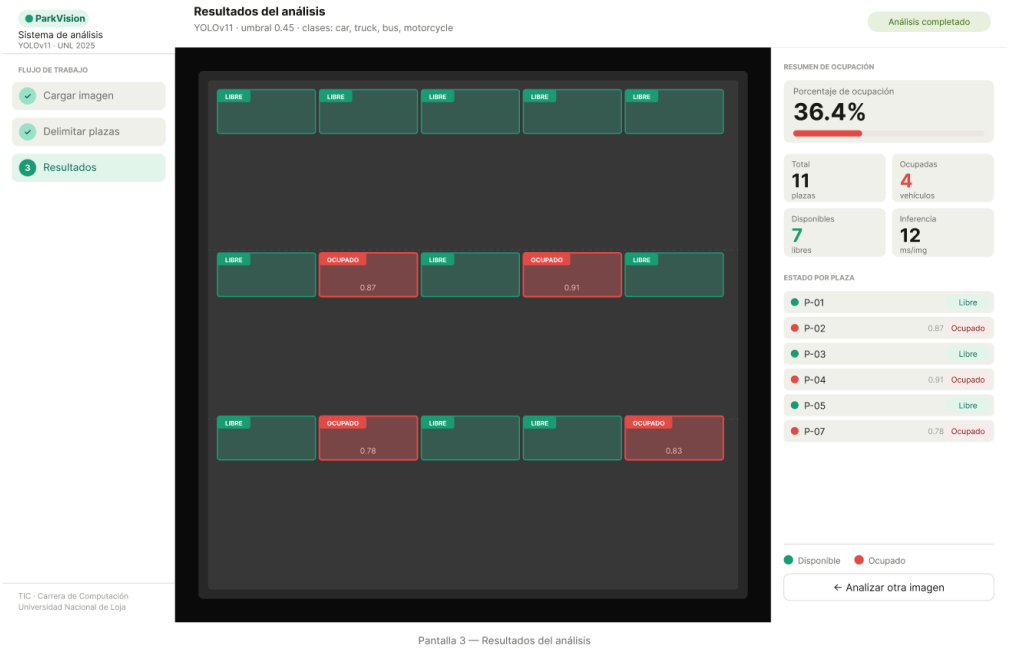

Figura 3: Visualización de resultados

3.1.2 Interfaces de comunicación

Esta sección describe los protocolos y mecanismos de comunicación utilizados por el prototipo para el intercambio de datos entre sus módulos internos y el servicio externo.

- Frontend - backend: mediante el protocolo HTTP bajo una arquitectura REST. El frontend envía solicitudes HTTP POST al backend con la imagen codificada y las coordenadas de las ROIs. El backend retorna una respuesta en formato JSON con los estados de ocupación por plaza, el porcentaje global de ocupación y las métricas calculadas.
- Backend - RoboFlow: mediante el protocolo HTTPS hacia el endpoint https://detect.roboflow.com/. El backend envía una solicitud HTTP POST con la imagen codificada en base64 y los parámetros de autenticación (api_key y confidence) . Roboflow retorna un objeto JSON con las coordenadas de cada plaza detectada.

3.2 Actores

Tabla 2. Actores del sistema

ID	Actor	Descripción	Requisitos asociados
AC1	Usuario	Persona que interactúa directamente con la interfaz web del prototipo SmartPark. Es responsable de cargar las imágenes de estacionamiento, delimitar manualmente las regiones de interés (ROIs) sobre el lienzo, iniciar el proceso de inferencia y consultar los resultados de ocupación.	RF1, RF2, RF5, RF6
AC2	Sistema	Componente back-end del prototipo, implementado con el framework Flask sobre Python. Actúa como actor sistema: recibe las solicitudes HTTP enviadas por el front-end (Next.js), ejecuta el procesamiento de la imagen (RF3), lanza la inferencia mediante el modelo YOLOv11 (RF4) y retorna los resultados de ocupación al cliente de forma autónoma, sin intervención humana adicional.	RF3, RF4, RF6

3.3 Requisitos Funcionales

Tabla 3. Requisitos funcionales

Id RF	Nombre del requisito	Descripción del requisito
RF1	Gestión de carga de imágenes	El sistema debe permitir al usuario la carga de imágenes de estacionamientos en formatos .webp, .jpg, .jpeg y .png, validando el tipo y tamaño del archivo.
RF2	Delimitación manual y automática de regiones de interés	El sistema debe permitir al usuario trazar y gestionar manualmente o automáticamente los polígonos (regiones de interés) que representan cada plaza de estacionamiento, a través de una interfaz gráfica sobre el lienzo.
RF3	Procesamiento de imágenes	El sistema debe ejecutar automáticamente el reescalado, la normalización de píxeles, el ajuste de contraste de la imagen y el filtro Gaussiano.
RF4	Análisis de ocupación mediante YOLOv11	El sistema debe procesar la imagen mediante el modelo YOLOv11 para detectar vehículos y determinar el estado (libre/ocupado) de cada región de interés definida.
RF5	Visualización de resultados	El sistema debe renderizar sobre la imagen original los estados de ocupación mediante capas de color y etiquetas informativas.
RF6	Reporte de métricas de ocupación	El sistema debe calcular y mostrar un resumen estadístico detallando el total de plazas, el conteo por estado y el porcentaje de uso.

3.3.1. Criterios de aceptación

Por cada requisito funcional se establecen los criterios de aceptación.

Tabla 4. Criterios de aceptación de los requisitos funcionales

Id RF	Criterios de aceptación
RF1	1. La interfaz debe incluir una zona de "Arrastrar y Soltar" (Drag & Drop) y un explorador de archivos.
	2. El sistema debe validar extensiones (.webp, .jpg, .jpeg, .png) y emitir un mensaje de error ante formatos no soportados.
	3. El sistema debe rechazar archivos mayores a 10MB
	4. Se debe mostrar una barra de progreso visual durante la subida.
	5. El usuario debe tener la opción de descartar la imagen y cargar una nueva.
	6. Tras una carga exitosa, el sistema debe habilitar un botón de navegación hacia el módulo de "Delimitación Manual de regiones de interés".
RF2	1. El sistema debe renderizar la imagen en un lienzo (canvas)

	2. El usuario debe permitir dibujar, editar posición y eliminar polígonos (regiones de interés) sobre el lienzo.
	3. Los polígonos trazados deben mostrarse con un borde de color neutro antes del procesamiento.
	4. El sistema debe permitir borrar un polígono mal trazado o resetear el lienzo por completo
	5. El botón "Ejecutar Inferencia" debe permanecer inhabilitado hasta que exista al menos un polígono definido.
	6. La interfaz debe contener un panel informativo que guíe al usuario en el proceso de delimitación manual, detallando las herramientas disponibles para el trazado de los espacios de estacionamiento.
	7. El sistema debe permitir marcar automáticamente las delimitaciones de los espacios de estacionamiento mediante un modelo pre entrenado de RoboFlow
RF3	1. El sistema debe redimensionar la imagen a 640x640 px mediante técnica de Letterboxing.
	2. Se debe aplicar el algoritmo CLAHE para mitigar efectos de sombras y un filtro Gaussiano para reducción de ruido digital.
RF4	1. El sistema debe ejecutar la inferencia con un umbral de confianza fijo de 0.45.
	2. Se deben filtrar únicamente las clases de la taxonomía de COCO: car, truck, bus, motorcycle.
	3. La lógica de ocupación se validará mediante la presencia del centroide del vehículo dentro de la ROI trazada en el RF2.
RF5	1. Las plazas deben teñirse con un overlay semitransparente: Verde (Disponible) y Rojo (Ocupado).
	2. Cada plaza debe mostrar un rótulo de texto con su estado y el porcentaje de confianza de la detección.
RF6	1. El panel de resumen debe mostrar: Total de plazas, plazas ocupadas y plazas disponibles.
	2. Se debe mostrar el porcentaje de ocupación con una precisión de un decimal.

3.4 Requisitos de Rendimiento.

3.4.1. Dataset de evaluación

Las métricas de rendimiento definidas en RNF1, RNF2 y RNF3 serán evaluadas sobre el dataset PKLot, un conjunto de datos público de referencia para la clasificación del estado de ocupación de estacionamientos, desarrollado por la Universidad Federal de Paraná, Brasil [1].

El dataset contiene 12.417 imágenes de resolución 1280×720 píxeles, capturadas desde dos estacionamientos distintos bajo tres condiciones climáticas: días soleados, nublados y lluviosos

3.4.2. Métricas de evaluación

Tabla 5. Requisitos de rendimiento

Id RNF	Categoría	Descripción del requisito	Criterio de aceptación
RNF1	Exactitud	El modelo debe alcanzar una exactitud superior al 90% en la clasificación del estado de ocupación de plazas.	Medido sobre el dataset PKLot, usando la fórmula (TP+TN)/total.
RNF2	Sensibilidad	El modelo debe mantener una sensibilidad superior al 90% en la detección de vehículos presentes.	Medido como TP/(TP+FN) sobre el mismo dataset.
RNF3	Precisión	El modelo debe alcanzar una precisión mínima del 90% en la detección de plazas ocupadas.	Medido como TP/(TP+FP) sobre el mismo dataset

4. Anexos

Anexo A. Trazabilidad de los requisitos

La trazabilidad entre los requisitos funcionales definidos en el presente documento y los componentes arquitectónicos que los implementan se encuentra documentada en el documento de arquitectura del sistema. Dicho documento forma parte del conjunto de artefactos del proyecto de integración curricular y puede consultarse en el repositorio del proyecto: https://github.com/ticestacionamiento/Smartpark.git